

Server-Factory-Core-Framework

Le moteur commun derrière chaque Server Factory.

Résumé

Core Framework est le framework Kotlin qui soutient la famille d'outils de provisionnement Server Factory. Il fournit le moteur commun et les abstractions sur lesquels s'appuient des projets comme Mail Server Factory, permettant à chaque « usine » de réutiliser une base éprouvée plutôt que de réimplémenter les primitives de provisionnement.

Description courte

Le framework Kotlin partagé à la base de l'écosystème Server Factory. Il fournit le moteur de provisionnement commun, les abstractions de connexion et la machinerie des étapes d'installation exploitées par les usines en aval (Mail Server Factory, Web Service Factory, SonarQube Factory, entre autres).

Description détaillée

Core Framework est cette pièce d'ingénierie discrète qui rend possible toute la famille Server-Factory : le moteur réutilisable sur lequel repose chaque produit « usine » (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory). L'approche Server Factory est déclarative — l'utilisateur décrit l'infrastructure souhaitée sous forme de configuration, et une usine interprète cette description pour installer et initialiser les logiciels sur un système cible. Core Framework est précisément l'endroit où réside la machinerie commune à ce modèle : les abstractions de connexion et de transport permettant d'atteindre tous les types de cibles, le modèle d'étapes d'installation qui définit *comment* le logiciel est provisionné, et l'ensemble des mécanismes partagés que chaque usine devrait autrement réécrire. Il apporte une réponse à une question structurelle que

toute chaîne d'outils multi-produits finit par se poser — où placer le moteur commun ? — et le fait de bien y répondre une fois pour toutes permet à la famille de rester cohérente plutôt que de se fragmenter en quatre provisionneurs aux comportements subtilement différents. En centralisant cette logique dans un seul framework Kotlin, la famille évite de dupliquer la logique de provisionnement entre les produits et garantit une cohérence comportementale : une amélioration d'un type de connexion ou d'une primitive d'installation dans Core Framework profite immédiatement à toutes les usines en aval. Il est presque entièrement écrit en Kotlin (environ 990 Ko de code Kotlin avec une fine couche en Shell), reflétant son rôle de bibliothèque plutôt que de collection de scripts. Les dépôts en aval y font référence comme dépendance canonique (Parallels-Utils, Qemu-Utils, Utils et les packs Definitions citent tous le dépôt Core Framework comme noyau de l'écosystème). Son fichier README est volontairement minimal — il s'agit d'une infrastructure destinée à d'autres projets, versionnée via `version.txt/version_code.txt` — et il précède les travaux ultérieurs de AI, ce qui en fait un élément du patrimoine mature des outils DevOps de l'organisation.

Pourquoi nous l'avons créé

Chaque outil de provisionnement a besoin des mêmes fondations : des moyens de se connecter aux cibles et des étapes pour installer/configurer les logiciels. Les réimplémenter pour chaque produit fragmenterait les comportements et multiplierait les bugs. Core Framework les centralise pour que toutes les usines partagent un moteur fiable.

Pourquoi c'est un atout majeur

C'est le point de levier le plus puissant de toute la famille : un type de connexion renforcé ou une primitive d'installation améliorée ici se répercute instantanément sur toutes les usines, garantissant cohérence et nouvelles capacités à l'ensemble de la chaîne d'outils. C'est la philosophie « concevoir une fois, réutiliser partout » appliquée là où elle rapporte le plus — au niveau de la couche fondatrice de l'automatisation des infrastructures, où une correction au bon endroit résout tout en aval.

Contenu

Ce qui est innovant

- Un cadre de provisionnement réutilisable unique qui abstrait la logique de connexion et des étapes d'installation.

- Une séparation nette entre le moteur (Cadre de base) et les fabriques spécifiques aux produits.
- Une distribution verrouillée par version (`version.txt/`
`version_code.txt`) pour une consommation reproductible.

Défis et solutions

- **Éviter la duplication de la logique de provisionnement** : résolu en extrayant les mécanismes partagés dans un cadre unique consommé par toutes les fabriques.
- **Comportement cohérent entre les produits** : résolu grâce à des abstractions communes, garantissant que les types de connexion et les étapes se comportent de manière identique partout.
- **(NON VÉRIFIÉ)** : les API internes spécifiques ne sont pas documentées dans le README public ; considérer les détails des interfaces comme non vérifiés au-delà du « cadre partagé consommé par les fabriques ».

Pile technologique (pourquoi et comment)

- **Kotlin** — l'intégralité du cadre (~990 Ko) ; le langage de la famille Server Factory.
- **Shell** — scripts de support minimalistes.
- **Gradle** — chaîne d'outils de compilation (cohérente avec l'utilisation de `./gradlew` par la famille).

Remarque : GitHub marque le dépôt comme un fork au sein de l'organisation Server-Factory. Non centré sur AI ; présenté comme l'épine dorsale de la chaîne d'outils de provisionnement.